lamae

BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 01a — Préambule et langage C

```
bash — R3.05
$ gcc -std=c17 -Wall bonjour.c -o bonjour
$ ./bonjour
Bonjour, système !
$ echo $?
0
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

Organisation du module

5	4	6
CM	**TD**	**TP**
cours magistraux	travaux dirigés	travaux pratiques

Ressource R3.05 — Programmation système

- 15 h TD + 15 h TP (programme national)
- Compétence 3 : administrer des systèmes informatiques communicants complexes
- Coefficient 22 % dans l'UE 3.3

01

Travail en binôme

9 séances de TD/TP : mettez-vous en binôme avec un·e camarade du même groupe de TP.

02

1 séance de TP noté

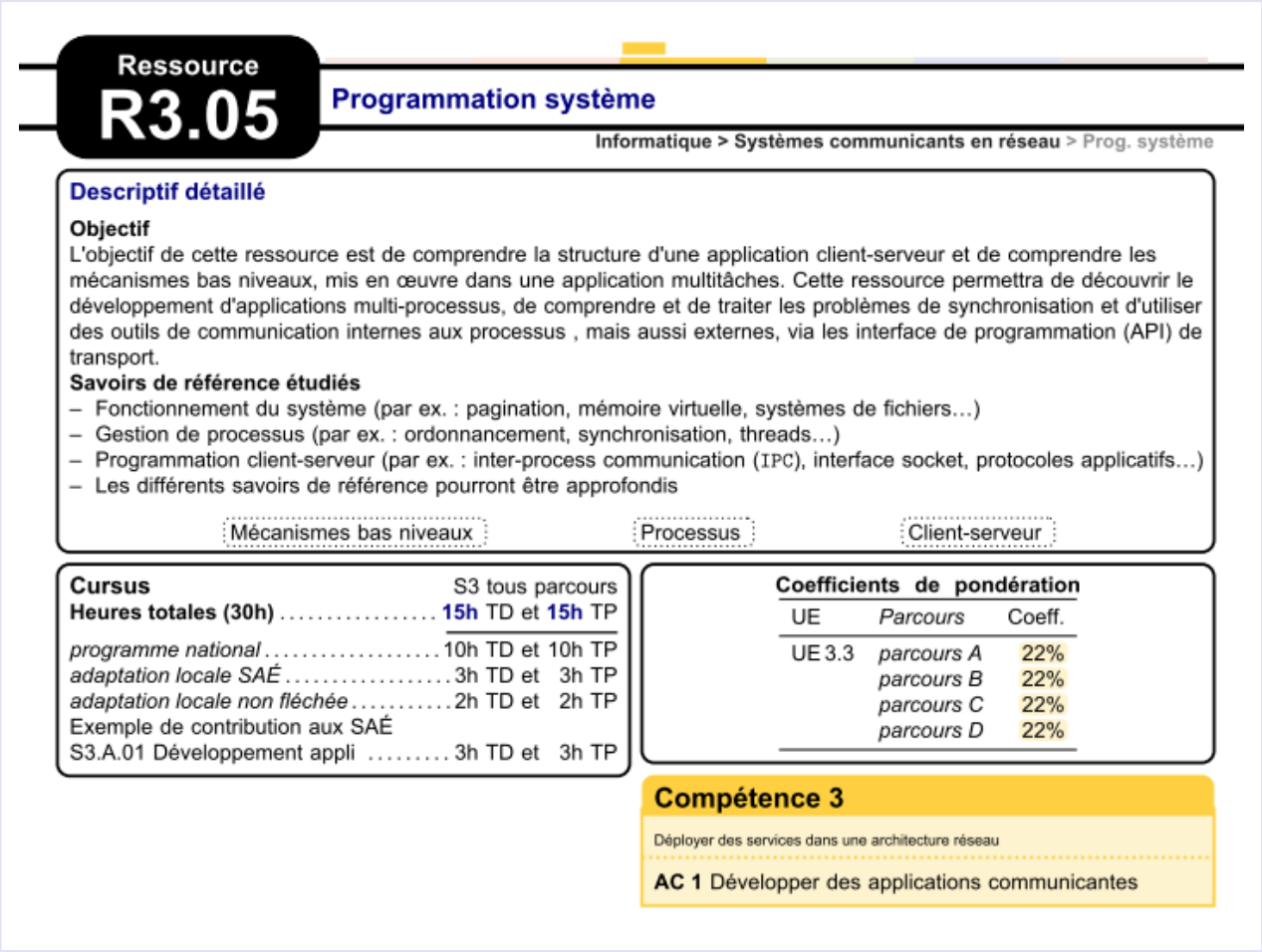
Une séance de TP est consacrée à une évaluation pratique.

03

Contrôle continu

L'évaluation se fait tout au long du module : TP rendus, TP noté, contrôles.

R3.05 dans le référentiel du BUT



Ressource
R3.05 Programmation système

Informatique > Systèmes communicants en réseau > Prog. système

Descriptif détaillé

Objectif
L'objectif de cette ressource est de comprendre la structure d'une application client-serveur et de comprendre les mécanismes bas niveaux, mis en œuvre dans une application multitâches. Cette ressource permettra de découvrir le développement d'applications multi-processus, de comprendre et de traiter les problèmes de synchronisation et d'utiliser des outils de communication internes aux processus , mais aussi externes, via les interface de programmation (API) de transport.

Savoirs de référence étudiés
- Fonctionnement du système (par ex. : pagination, mémoire virtuelle, systèmes de fichiers…)
- Gestion de processus (par ex. : ordonnancement, synchronisation, threads…)
- Programmation client-serveur (par ex. : inter-process communication (IPC), interface socket, protocoles applicatifs…)
- Les différents savoirs de référence pourront être approfondis

Mécanismes bas niveaux | Processus | Client-serveur

Cursus	S3 tous parcours
Heures totales (30h)	**15h** TD et **15h** TP
programme national	10h TD et 10h TP
adaptation locale SAÉ	3h TD et 3h TP
adaptation locale non fléchée	2h TD et 2h TP
Exemple de contribution aux SAÉ	
S3.A.01 Développement appli	3h TD et 3h TP

Coefficients de pondération

UE	*Parcours*	Coeff.
UE 3.3	*parcours A*	22%
	parcours B	22%
	parcours C	22%
	parcours D	22%

Compétence 3
Déployer des services dans une architecture réseau
AC 1 Développer des applications communicantes

Objectif : comprendre la structure d'une application client-serveur et les mécanismes bas niveau d'une application multitâche.

01

Mécanismes bas niveau

Pagination, mémoire virtuelle, systèmes de fichiers

02

Processus

Ordonnancement, synchronisation, threads

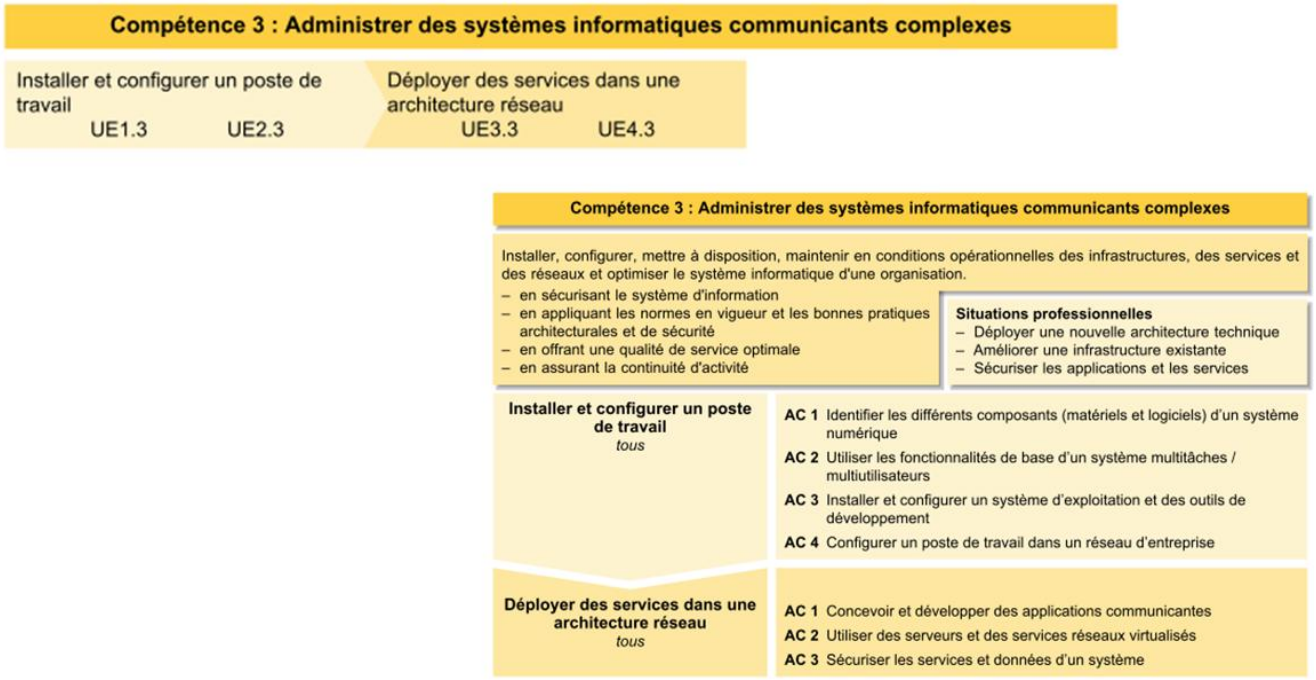
03

Client-serveur

Communication inter-processus (IPC), sockets

Compétence 3 et poids dans les UE

Compétence 3 — Administrer des systèmes communicants complexes



Ressources et SAÉ du semestre 3

Ressource/SAÉ	UE3.1	UE3.2	UE3.3	UE3.4	UE3.5	UE3.6
SAÉ	**40**	**40**	**40**	**40**	**40**	**40**
S3.A.01 Développement appli	40	40	40	40	40	40
Portfolio			*Non évalué ce semestre*			
P3.01 Portfolio			—			
Ressources	**60**	**60**	**60**	**60**	**60**	**60**
R3.01 Développement web	15	5	5	10		
R3.02 Dév. efficace	10	13				
R3.03 Analyse	12	5			10	
R3.04 Qualité de développement	15				8	5
R3.05 Prog. système			22			
R3.06 Archi. Réseaux		5	18			
R3.07 SQL et programmation				25		
R3.08 Probabilités		17		5		
R3.09 Cryptographie		10	10	5		
R3.10 Management SI				10	18	16
R3.11 Droit contrats et num.	8			5	10	
R3.12 Anglais		5	5		7	8
R3.13 Comm. pro.					7	16
R3.14 PPP						15
Total	**100**	**100**	**100**	**100**	**100**	**100**

R3.05 pèse 22 % de l'UE 3.3

Les 9 séances de TD/TP

Le langage C | **Le système : fichiers, processus, communication**

1	2	3	4	5	6	7	8	9
Langage C partie 1	Langage C partie 2	Les pointeurs	L'allocation dynamique	Les fichiers	Les processus	Les tubes	Threads et mutex	Les sockets

Ce CM 01 prépare les séances 1 à 4. Il est découpé en deux parties : **CM 01a** (système, langage C, types, pointeurs) et **CM 01b** (entrées-sorties standard, allocation dynamique).

01

Le système d'exploitation

Le chef d'orchestre entre les applications et le matériel

Les 5 rôles du système d'exploitation

1 Processus

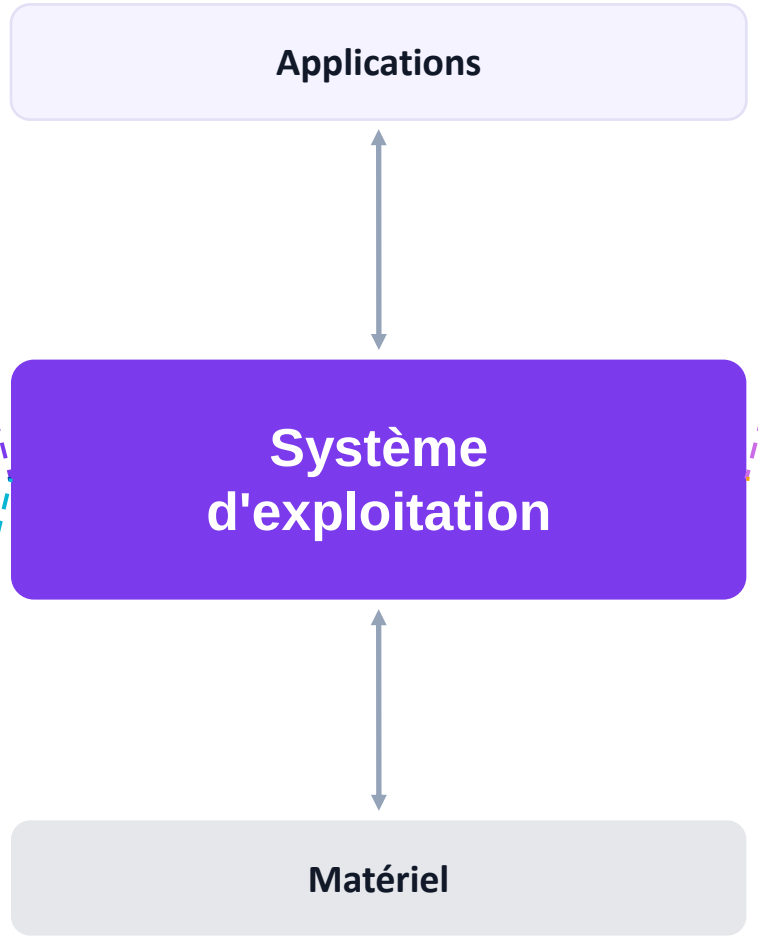
Exécuter les applications, leur donner des ressources, les arrêter

2 Mémoire

Répartir et protéger la RAM ; mémoire virtuelle

3 Fichiers

Lire, écrire, organiser les fichiers et leurs droits



4 Entrées / sorties

Accéder au matériel par l'intermédiaire des pilotes

5 Réseau

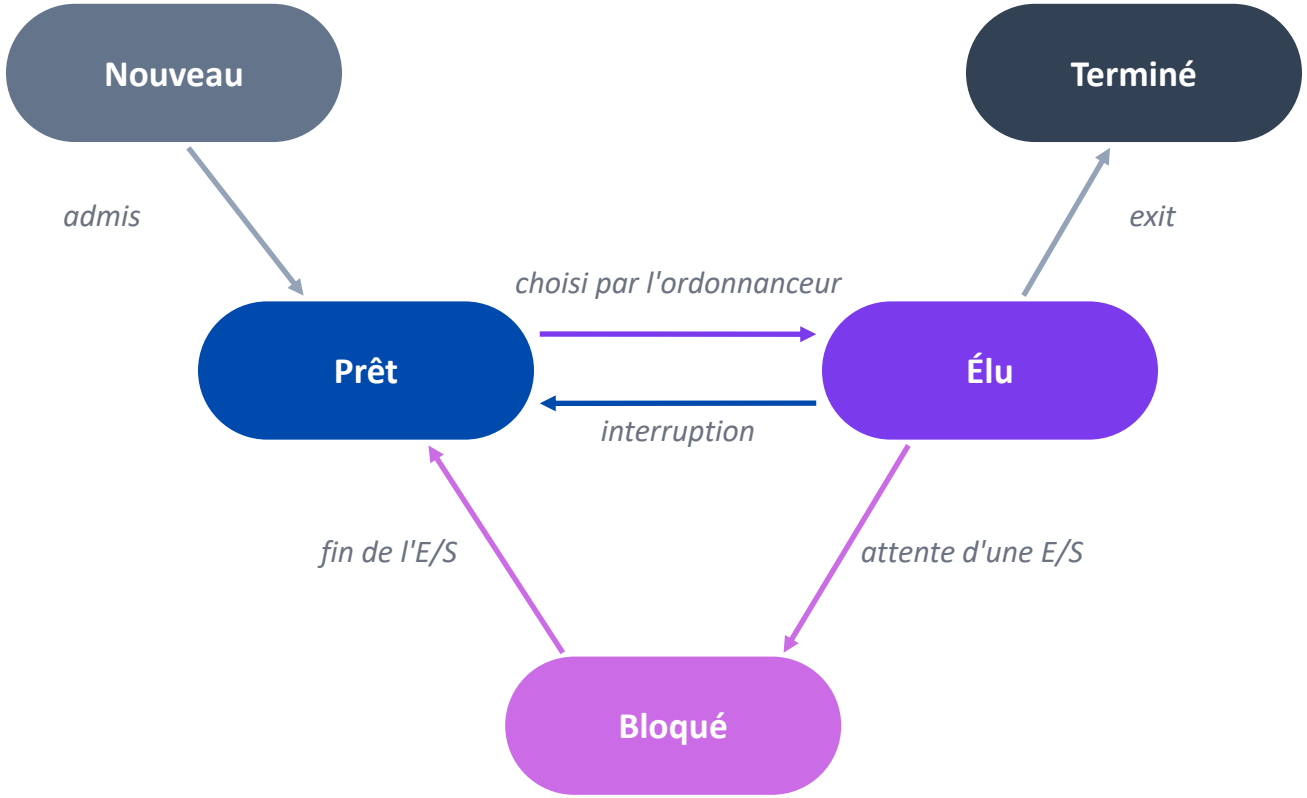
Échanger des données avec d'autres machines

Comment un programme demande-t-il ces services ?
Par des appels système

Rôle 1 · Gérer l'exécution des processus

1 Les processus

- Un processus = un programme en cours d'exécution
- L'OS lui affecte les ressources nécessaires : temps CPU, mémoire, fichiers…
- L'ordonnanceur choisit à chaque instant quel processus s'exécute
- L'OS peut tuer une application qui ne répond plus (kill)



Rôle 2 · Gérer la mémoire

2 La mémoire

- Gérer l'espace mémoire alloué à chaque application (et à chaque utilisateur)
- Isoler les processus : l'un ne peut pas écrire dans la mémoire d'un autre
- Si la RAM manque, l'OS utilise une zone du disque : la mémoire virtuelle (swap)

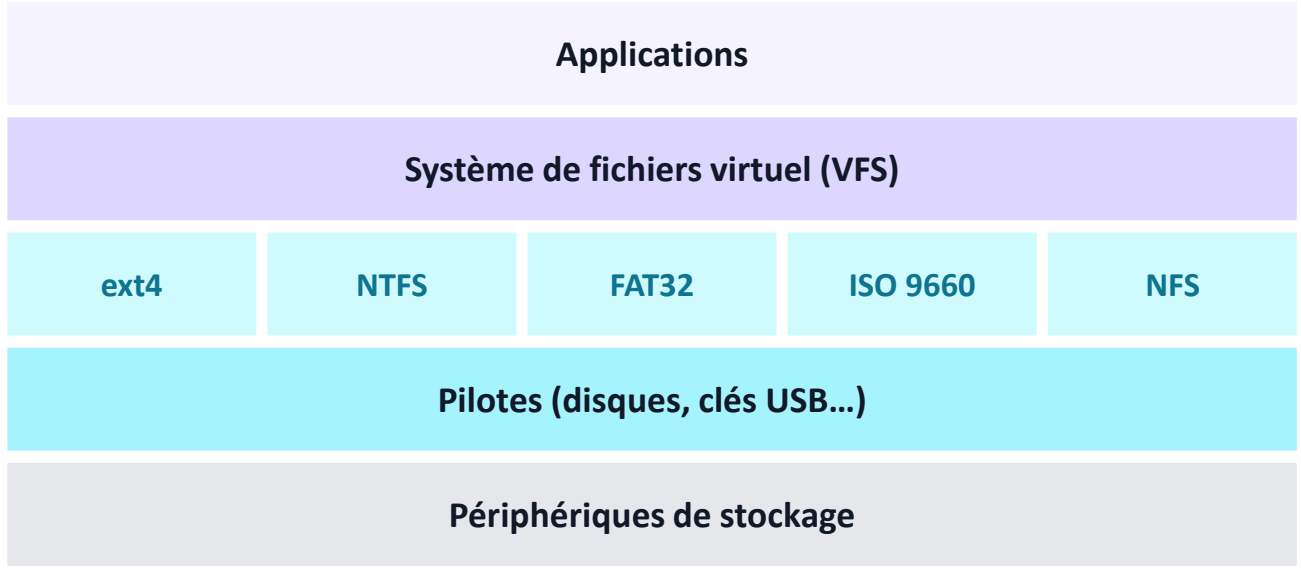


Fragmentation : de petits trous inutilisables apparaissent entre les processus.

Rôle 3 · Gérer les fichiers

3 Les fichiers

- Lire et écrire dans le système de fichiers
- Organiser les fichiers en une arborescence de répertoires
- Contrôler les droits d'accès des utilisateurs et des applications



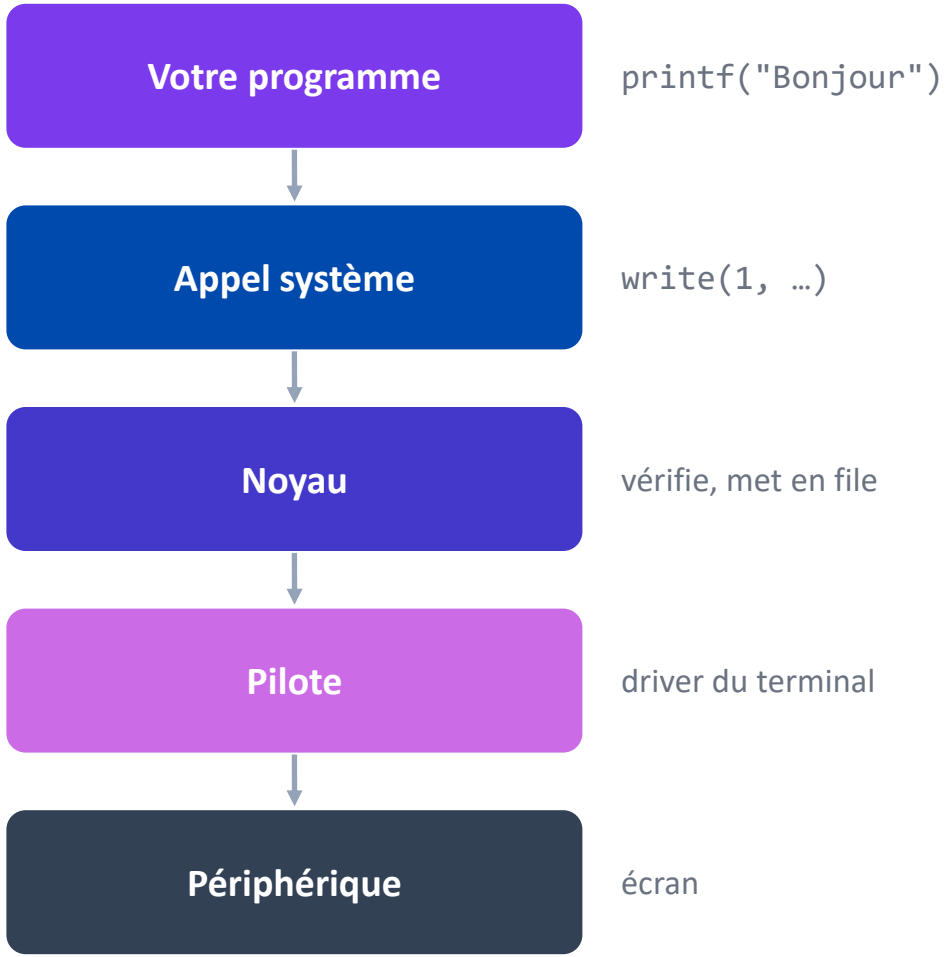

```
terminal
$ ls -l essai.c
-rw-r--r-- 1 cruz info 1024 sept. 8 essai.c
```

`rw-` propriétaire `r--` groupe `r--` les autres

Rôle 4 · Gérer les entrées / sorties

4 Les entrées / sorties

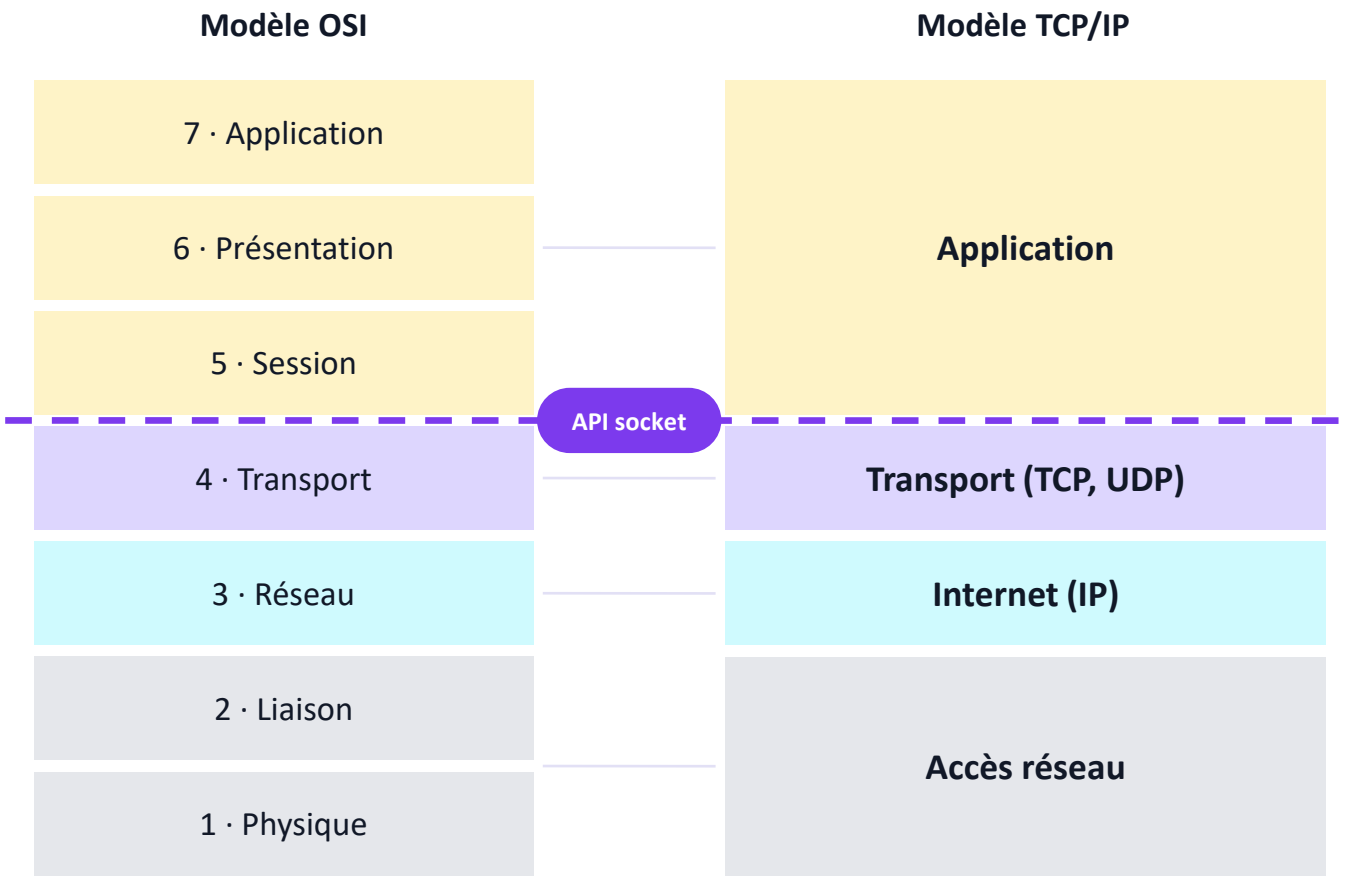
- Gérer l'accès des programmes aux ressources matérielles : clavier, écran, disque, réseau…
- L'accès passe par des pilotes (drivers) fournis au noyau
- Un programme ne parle jamais directement au matériel : il demande au noyau



Rôle 5 · Gérer le réseau

5 Le réseau

- Permettre l'échange de données entre la machine locale et une machine distante
- Le noyau implémente les protocoles (TCP/IP)
- Les programmes y accèdent par les sockets (séance 9)



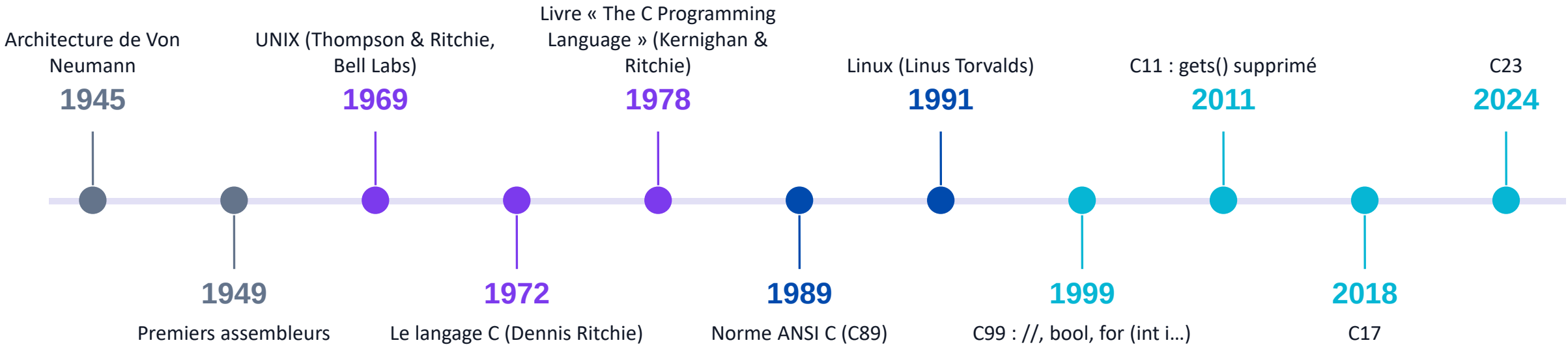
au-dessus : votre programme · en dessous : le noyau

02

Pourquoi le langage C ?

Un langage proche de la machine, conçu pour écrire des systèmes

Une brève histoire



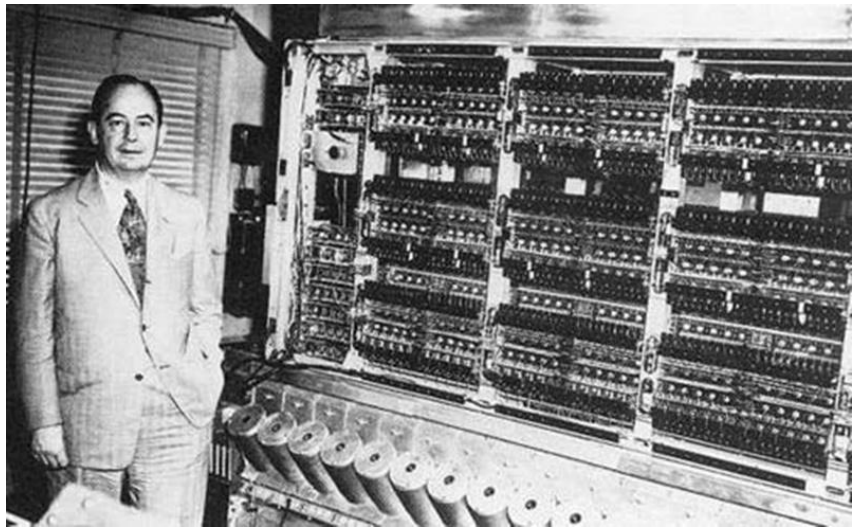
Portable

Un même source se compile sur x86, ARM, RISC-V… alors que l'assembleur est propre à chaque processeur.

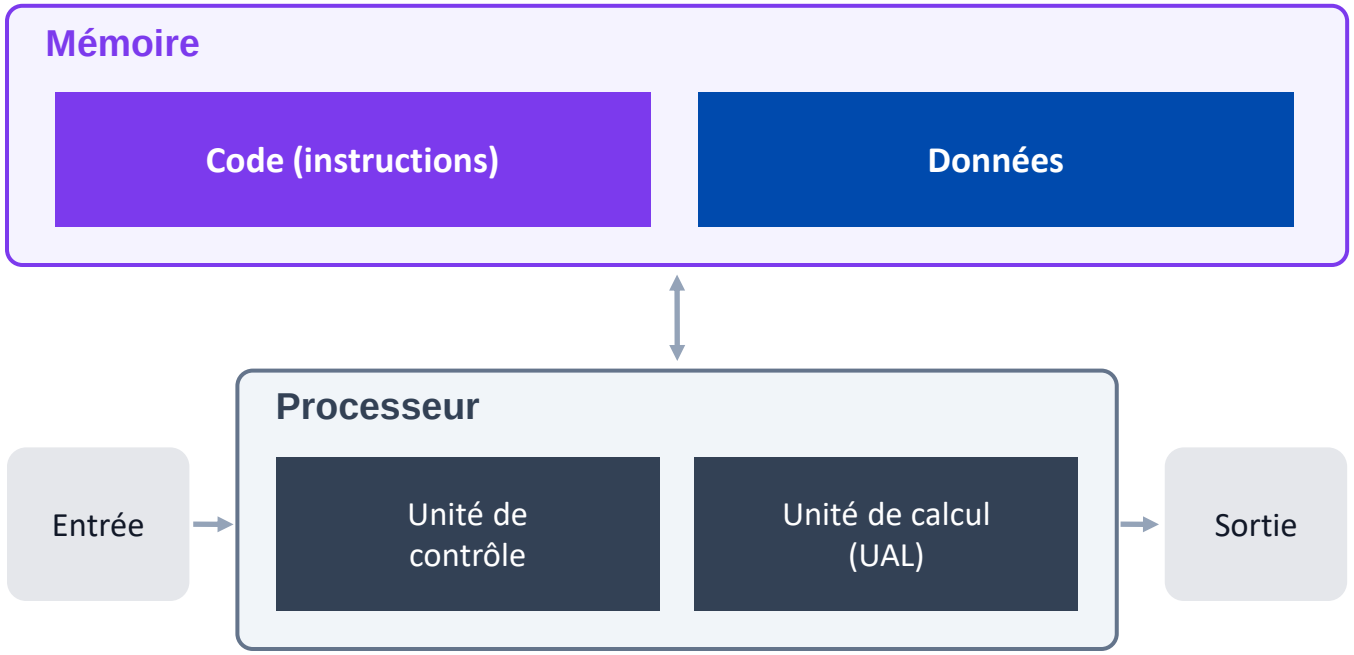
Proche de la machine

Accès direct à la mémoire (pointeurs), pas de ramasse-miettes : c'est le programmeur qui gère.

L'architecture de Von Neumann (1945)



John von Neumann devant l'IAS machine



Le programme est rangé en mémoire comme les données.
Tout a donc une adresse : variables, tableaux, fonctions… C'est ce qui rend possibles les pointeurs du C — et certaines failles (écrire hors d'un tableau peut écraser autre chose).

L'assembleur : un langage par architecture

- Premier langage « symbolique » : un mnémonique (mov, add…) = une instruction machine
- Appelle le système directement : interruptions BIOS/DOS (int 21h), ou syscall sous Linux
- Un assembleur par processeur : x86, ARM, RISC-V…

bonjour.s — x86-64

```
section .data
msg:    db "Bonjour", 10

section .text
global _start
_start:
    mov rax, 1          ; write
    mov rdi, 1          ; stdout
    mov rsi, msg
    mov rdx, 8
    syscall
    mov rax, 60         ; exit
    xor rdi, rdi
    syscall
```

bonjour.c

```
#include <unistd.h>

int main(void)
{
    write(1, "Bonjour\n", 8);
    return 0;
}
```

Même programme en C
Plus court, lisible, et il se compile sur n'importe quel processeur.

C, UNIX et Linux



> 90 %

du noyau UNIX / Linux est écrit en C

Depuis 2022, Rust entre aussi dans le noyau Linux, encore à la marge.

01

Le compilateur

Le compilateur C (gcc) est écrit en grande partie en C, ou avec des outils qui génèrent du C.

02

La chaîne de compilation

Même chose pour les autres outils : préprocesseur, assembleur, éditeur de liens.

03

Les utilitaires du système

Le shell et les commandes (ls, cp, grep…) sont aussi écrits en C.

La bibliothèque standard du C

<string.h>

Chaînes de caractères

`strlen, strcpy, strcmp, strcat…`

<stdio.h>

Entrées-sorties

`printf, scanf, fopen, fgets…`

<stdlib.h>

Mémoire dynamique et utilitaires

`malloc, free, exit, atoi…`

<unistd.h>

Accès au système (POSIX)

`read, write, fork, sleep…`

<unistd.h> ne fait pas partie de la norme C : c'est l'interface POSIX des systèmes de type UNIX.

bonjour.c

```c
#include <stdio.h>   // déclare printf (son prototype)
int main(void) {
    printf("Bonjour\n");   // le code de printf est dans la libc
    return 0;
}
```

`#include` recopie seulement les déclarations. Le code des fonctions est dans la bibliothèque (glibc), ajoutée à l'édition de liens.

Où se situe votre programme ? Architecture de Linux



03

Du source à l'exécutable

Fichiers .c, fonction main et chaîne de compilation

Structure d'un programme C

- Le code est réparti dans un ou plusieurs fichiers texte .c (et des en-têtes .h), ex. monprogramme.c
- Un programme C est un ensemble de fonctions
- L'une d'elles doit s'appeler main : c'est le point d'entrée de l'exécutable

bonjour.c

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Bonjour, système !\n");
    return 0;
}
```

Deux écritures équivalentes

```c
int main(int argc, char *argv[])
int main(int argc, char **argv)
```

argc
nombre d'arguments, nom du programme compris

argv
tableau de chaînes : les arguments

return 0
code de retour : 0 = succès (echo $? dans le shell)

La chaîne de compilation



Votre code : fonctions, #include, commentaires…

1. préprocesseur
remplace #include, #define…

`gcc -E`

2. compilateur
traduit le C en assembleur

`gcc -S`

3. assembleur
produit le langage machine

`gcc -c`

4. éditeur de liens
ajoute la libc et le code de démarrage

`gcc essai.o`

En pratique, gcc enchaîne les 4 étapes

```
$ gcc -c essai.c          # étapes 1 à 3  ->  essai.o
$ gcc essai.o -o essai    # étape 4 : édition de liens
$ gcc essai.c -o essai    # ou tout en une seule fois
$ ./essai
```

Les options de gcc

Chaque compilateur ajoute ses propres extensions à la norme ISO du C. On peut les désactiver pour garantir un code portable.

Option	Rôle
`-std=c17`	Respecter la norme C17 (l'ancien -ansi correspond à C89)
`-Wall -Wextra`	Activer un grand nombre d'avertissements
`-pedantic`	Rejeter ce qui ne respecte pas strictement la norme
`-Werror`	Transformer tous les avertissements en erreurs
`-g`	Ajouter les informations de débogage (gdb, valgrind)
`-O2`	Optimiser le code produit
`-fsanitize=address`	Détecter à l'exécution les accès mémoire invalides

En TP

La ligne de compilation à utiliser

```
gcc -std=c17 \
    -Wall -Wextra \
    -pedantic -g \
    essai.c -o essai
```

Un avertissement est presque toujours un bug en devenir : lisez-les et corrigez-les !

04

Syntaxe et structures de contrôle

Très proche de Java et du C++, qui en ont hérité

Syntaxe des fonctions

```
int  readBlock  (int deplacement, int type)  { … }
```

type de retour — nom de la fonction — liste des paramètres — corps : le code

Forme générale : `type_de_retour nom_fonction(liste_des_paramètres) { code }`

Déclarer avant d'utiliser

Toute fonction doit être déclarée avant son appel : définition placée avant, ou prototype : int readBlock(int, int);

Comme en Java / C++

Mêmes structures de contrôle et mêmes opérateurs (+ - / % == != && || …), à quelques restrictions près.

Déclarer les variables

En C89 : en début de bloc uniquement. Depuis C99 : n'importe où, y compris for (int i = 0; …).

Aucune initialisation automatique

Une variable locale non initialisée a une valeur indéterminée : initialisez toujours vos variables !

Un exemple commenté

exemple.c

```c
#include <stdio.h>

int une_variable_globale;

void fait_pas_grand_chose(void);

int main(int argc, char **argv)
{
    if (une_variable_globale == 3)
        fait_pas_grand_chose();
    return 0;
}

void fait_pas_grand_chose(void)
{
    char c = 7;
    une_variable_globale = (int) c;
}
```

Variable globale
visible dans tout le fichier, initialisée à 0 par défaut

Prototype
la fonction est déclarée avant d'être appelée dans main

Variable locale
n'existe que dans la fonction ; à initialiser soi-même

Cast (int)
conversion explicite de type : char → int

Les tests : if / else

tests.c

```c
if (expression)
    instruction;

if (expression)
    instruction1;
else
    instruction2;

if (note >= 16)
    printf("Très bien\n");
else if (note >= 12)
    printf("Bien\n");
else if (note >= 10)
    printf("Passable\n");
else
    printf("Ajourné\n");
```

Vrai ou faux ?

Une expression est fausse si elle vaut 0, vraie dans tous les autres cas.

`if (n)` ≡ `if (n != 0)`

Piège : = n'est pas ==

`if (x = 0)` affecte 0 à x, puis teste 0 : toujours faux !

`if (x == 0)` compare x à 0

gcc -Wall vous prévient : « suggest parentheses around assignment »

Les boucles

while

```c
int i = 0;
while (i < 3) {
    printf("%d\n", i);
    i++;
}
```

Test avant : 0 à n tours

do … while

```c
int n;
do {
    printf("Note ? ");
    scanf("%d", &n);
} while (n < 0 || n > 20);
```

Test après : au moins 1 tour

for

```c
for (int i = 0; i < 3; i++) {
    printf("%d\n", i);
}
```

Initialisation ; test ; incrément

do … while : on boucle tant que la condition est vraie. Idéal pour redemander une saisie tant qu'elle est invalide.

L'instruction switch

menu.c

```c
switch (choix) {
case 1:
    printf("Ouvrir\n");
    break;
case 2:
    printf("Enregistrer\n");
    /* break oublié ! */
case 3:
    printf("Quitter\n");
    break;
default:
    printf("Choix inconnu\n");
    break;
}
```

- L'expression est entière : int, char, enum
- Chaque case est une constante
- break termine le switch
- default : facultatif, traite les autres cas

Exécution avec choix = 2

```
Enregistrer
Quitter
```

Sans break, l'exécution « tombe » dans le case suivant.

Les ruptures de séquence

01

`break;`

Sort du bloc do, for, while ou switch le plus proche.

02

`continue;`

Passe directement à l'itération suivante de la boucle la plus proche.

03

`goto label;`

Saut inconditionnel vers « label: instruction ». À éviter : code illisible.

04

`return [expression];`

Quitte la fonction et rend la main à l'appelant, avec une valeur si le type de retour n'est pas void.

05

Les données en mémoire

Types, tableaux, structures et pointeurs

Les types élémentaires

Type	Octets*	Plage / précision
`char`	1	-128 … 127 (ou 0 … 255 selon la plateforme)
`unsigned char`	1	0 … 255
`short`	2	-32 768 … 32 767
`unsigned short`	2	0 … 65 535
`int`	4	-2 147 483 648 … 2 147 483 647
`unsigned int`	4	0 … 4 294 967 295
`long`	8	≈ ± 9,2 × 10^{18}
`float`	4	≈ 7 chiffres significatifs, jusqu'à ± 3,4 × 10^{38}
`double`	8	≈ 15 chiffres significatifs, jusqu'à ± 1,8 × 10^{308}
`long double`	16	80 bits utiles sur x86-64

** Tailles typiques sous Linux 64 bits avec gcc. La norme ne fixe que des minimums.*

int ≠ short

Dans les vieux cours, « int est un short ou un long selon le compilateur ». Aujourd'hui, int fait 4 octets partout.

Taille exacte ?

Utilisez sizeof, ou <stdint.h> quand la taille compte : int8_t, int32_t, uint64_t…

Les caractères sont des entiers



Quelques codes ASCII

'\0'	'\n'	' '	'0'	'9'	'A'	'Z'	'a'	'z'
0	10	32	48	57	65	90	97	122

Pas de type « chaîne » en C

Une chaîne est un tableau de char terminé par le caractère nul '\0'.

caracteres.c

```c
char c = 'A';
printf("%c = %d\n", c, c);   // A = 65

c = c + 1;
printf("%c\n", c);           // B

char m = 'G' + ('a' - 'A'); // 'g'
int  d = '7' - '0';         // 7
```

Les booléens

En C, une condition est fausse quand elle vaut 0, et vraie dans tous les autres cas.

`0 → faux` `1 → vrai` `-3 → vrai` `42 → vrai` `'A' → vrai` `NULL → faux`

Avant C99

```c
#define FALSE 0
#define TRUE  1

/* ou bien */
enum bool { FALSE, TRUE };
```

Depuis C99

```c
#include <stdbool.h>

bool fini = false;
while (!fini) {
    /* ... */
}
```

En C23, bool, true et false deviennent des mots-clés : l'#include n'est plus nécessaire.

Mesurer avec sizeof

taille.c

```c
#include <stdio.h>

int main(void)
{
    printf("char   : %zu\n", sizeof(char));
    printf("int    : %zu\n", sizeof(int));
    printf("long   : %zu\n", sizeof(long));
    printf("double : %zu\n", sizeof(double));
    printf("int *  : %zu\n", sizeof(int *));
    return 0;
}
```

sizeof(type) ou sizeof variable : le résultat est en octets.

terminal

```
$ gcc -Wall taille.c -o taille
$ ./taille
char   : 1
int    : 4
long   : 8
double : 8
int *  : 8
```

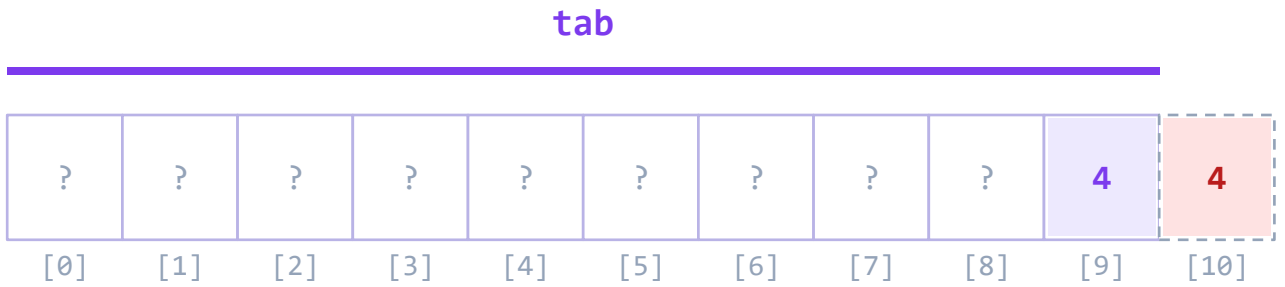
À savoir

sizeof est calculé à la compilation et renvoie un size_t (format %zu). Sur une machine 64 bits, un pointeur fait 8 octets, quel que soit le type pointé.

Les tableaux

- Zone de mémoire contiguë contenant des valeurs d'un même type
- Taille fixée à la définition : elle ne peut plus changer
- Indices de 0 à taille − 1
- Aucune vérification de dépassement : pas de marqueur de fin

```c
int tab[10];      // 10 entiers, valeurs indéterminées
tab[8 + 1] = 4;   // tab[9] : le dernier élément
tab[10] = 4;      // ERREUR : hors du tableau !
```



`tab[10] = 4;` compile sans erreur… mais écrase une autre donnée : c'est un comportement indéfini.

Initialiser tableaux et chaînes

init.c

```c
int v[] = {0, 1, 2};          // taille déduite : 3

float m[3][2] = {{0.0, 0.1},
                 {1.0, 1.1},
                 {2.0, 2.1}};
/* équivalent : {0.0, 0.1, 1.0, 1.1, 2.0, 2.1} */

char s1[20] = {'c','o','u','c','o','u','\0'};
char s2[20] = "coucou\n";     // '\0' ajouté
```

m en mémoire : ligne par ligne

0.0	0.1	1.0	1.1	2.0	2.1
[0][0]	[0][1]	[1][0]	[1][1]	[2][0]	[2][1]

s2 en mémoire

'c'	'o'	'u'	'c'	'o'	'u'	'\n'	'\0'	?	?	…
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	

`'\0'` marque la fin de la chaîne : prévoir une case de plus !

Les structures

Une structure est un type composé hétérogène : chaque champ a son propre type.

syntaxe

```
struct nom_structure {
    type_1 champ_1;
    ...
    type_n champ_n;
};

struct nom_structure article;    // variable
article.champ_i                  // accès à un champ
```

point.c

```
struct point {
    float x;
    float y;
};

struct point origine = {0.0, 0.0};
origine.y = 12.0;

typedef struct point Point;    // alias
Point a = {1.5, 2.0};
```

origine



typedef donne un nom plus court au type : Point au lieu de struct point.

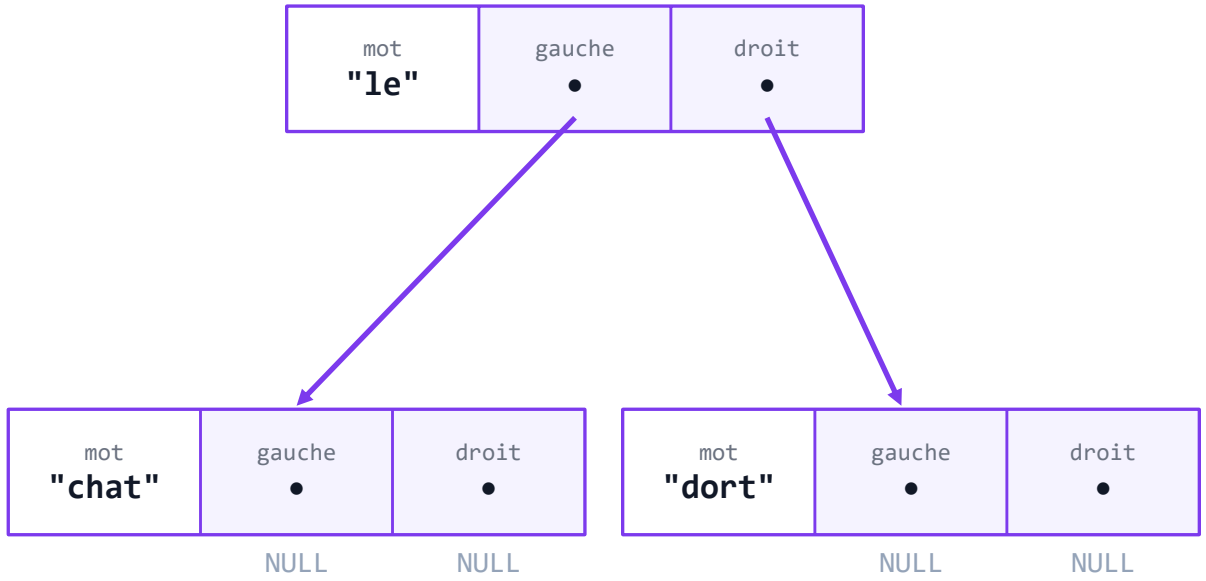
Pointeur sur structure et auto-référencement

noeud.c

```c
struct point *pp = &origine;
pp->x = 10.0;      // équivaut à (*pp).x = 10.0

struct noeud {
    char          *mot;
    struct noeud *gauche;
    struct noeud *droit;
};
```

Auto-référencement : un nœud contient des pointeurs vers des nœuds du même type. C'est la base des listes chaînées et des arbres (avec l'allocation dynamique, CM 01b).



Un arbre binaire de mots

Les pointeurs

Un pointeur est une variable dont le contenu est une **adresse mémoire**.

Il est défini en fonction du type de la donnée rangée à cette adresse :

```
char *pc;    // pointeur de caractère
int  *pi;    // pointeur d'entier
```

&

Opérateur « adresse de »

`&c` donne l'adresse mémoire de c

*

Opérateur « indirection »

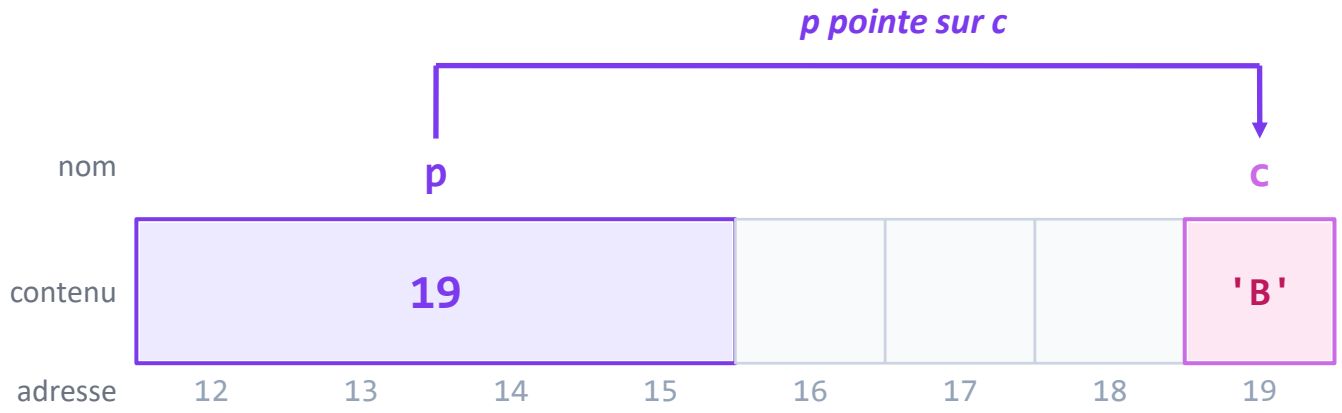
`*p` donne la valeur rangée à l'adresse contenue dans p

Les pointeurs pas à pas

pointeur.c

```c
char  c;
char *p;

c = 'A';
p = &c;
*p = 'B';
```



`p` et `&c` valent 19

`*p` et `c` valent `'A'` (65)

Ici, une adresse tient sur 4 cases pour simplifier (8 octets sur une machine 64 bits).

`*p = 'B';`
On écrit à l'adresse contenue dans p : c'est c qui est modifié, sans jamais écrire « c » !

Exercice : que valent x, y et pi ?

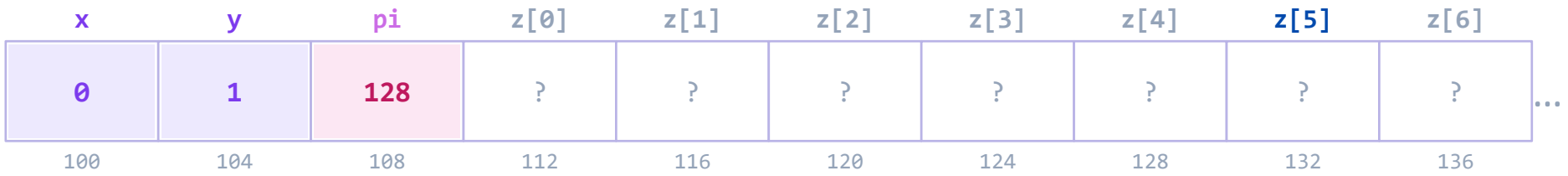
exercice.c

```c
int x = 1;
int y = 2;
int z[10];
int *pi;

pi = &x;
y = *pi;
*pi = 0;
pi = &z[5];
```

← pi pointe sur x
← y vaut 1
← x vaut 0
← pi pointe sur z[5]

Règle
Un pointeur ne peut pointer que sur un objet du type pour lequel il a été déclaré.
`int * → int`



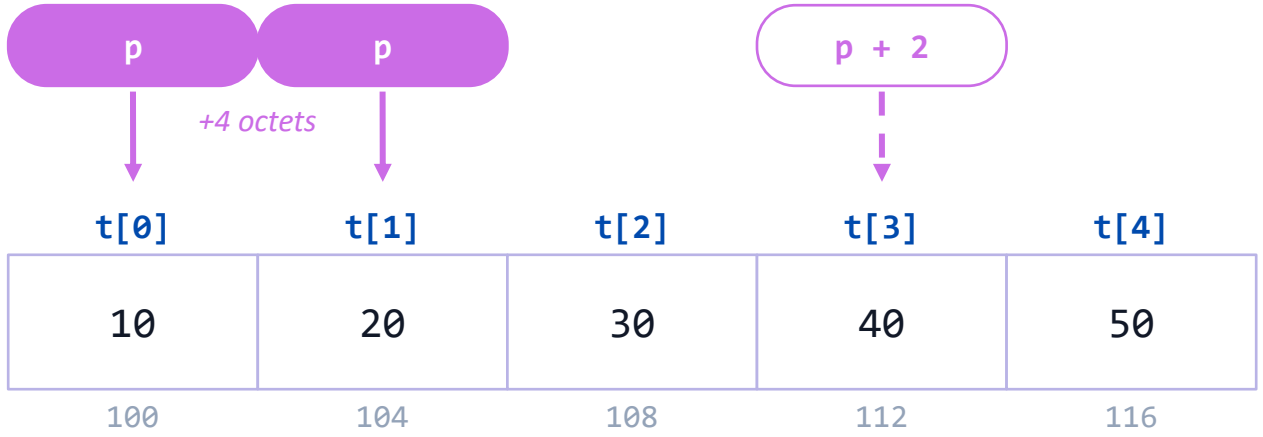
Adresses fictives : un int occupe 4 octets.

Pointeurs et tableaux

tableau.c

```c
int t[5] = {10, 20, 30, 40, 50};
int *p = &t[0];     // ou simplement : p = t;
int x = *p;         // x vaut 10

p = p + 1;          // avance de sizeof(int) = 4 octets
x = *(p + 2);       // x vaut 40, comme p[2] ou t[3]
```



t[i] ≡ *(t + i)

Le nom d'un tableau se convertit en l'adresse de son premier élément.

Arithmétique des pointeurs

p + 1 avance d'un élément, c'est-à-dire de sizeof(*p) octets.

06

Paramètres et arguments

Passage par valeur, passage par adresse, argc et argv

Passage par valeur

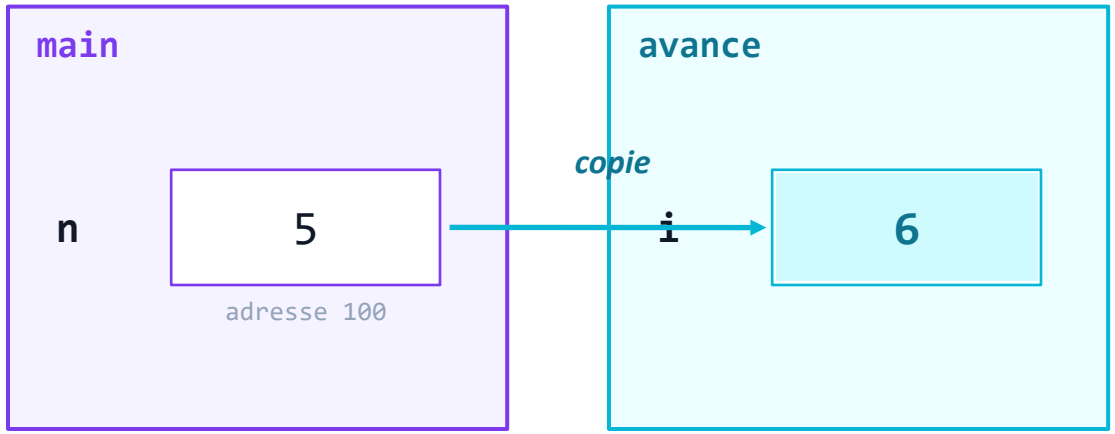
valeur.c

```c
#include <stdio.h>

void avance(int i)
{
    i = i + 1;
}

int main(void)
{
    int n = 5;
    avance(n);
    printf("%d\n", n);
    return 0;
}
```

la pile d'exécution




```
$ ./valeur
5
```

La fonction travaille sur une copie : n n'est pas modifié.

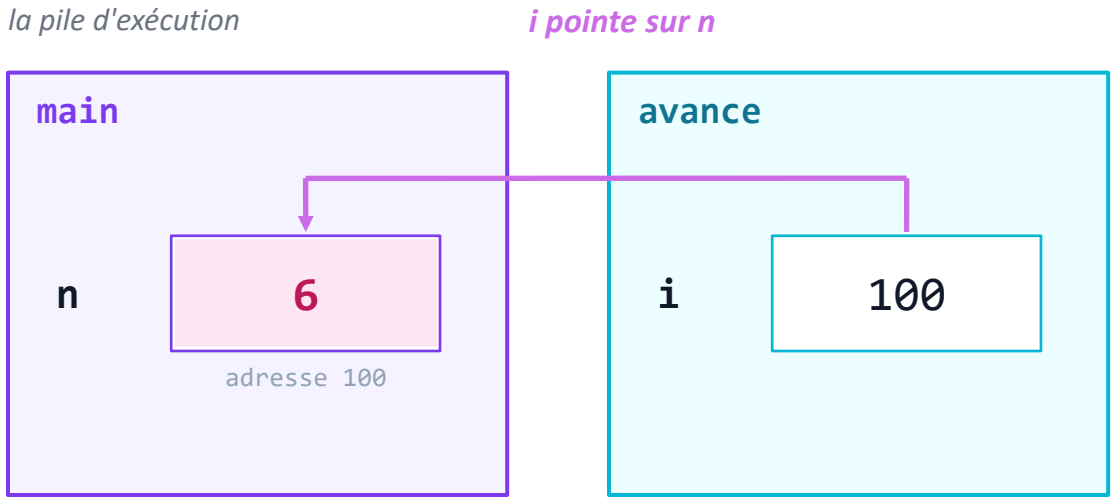
Passage par adresse

adresse.c

```c
#include <stdio.h>

void avance(int *i)        // déclaration
{
    *i = *i + 1;           // manipulation
}

int main(void)
{
    int n = 5;
    avance(&n);            // adresse de n
    printf("%d\n", n);
    return 0;
}
```




```
$ ./adresse
6
```

C'est pour cela que l'on écrit **`scanf("%d", &n)`** : scanf a besoin de l'adresse de n pour le modifier.

Arguments de la ligne de commande

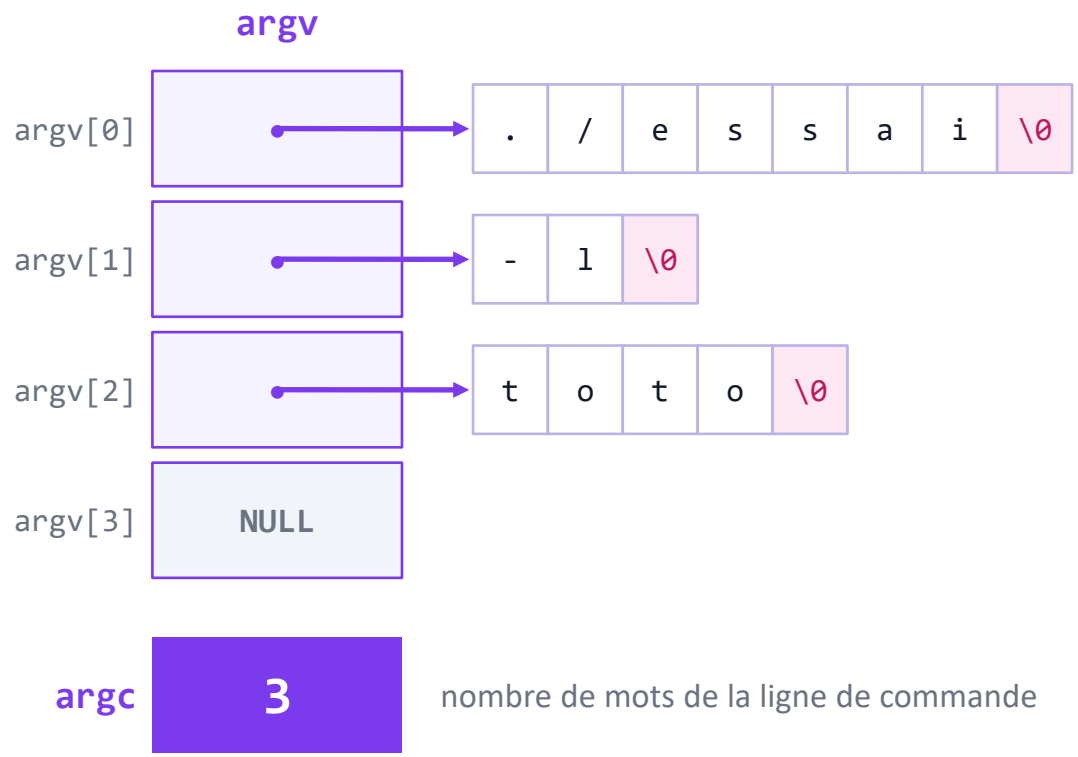
```
$ ./essai -l toto
```

essai.c

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    for (int i = 0; i < argc; i++)
        printf("argv[%d] = %s\n", i, argv[i]);
    return 0;
}
```

```
$ ./essai -l toto
argv[0] = ./essai
argv[1] = -l
argv[2] = toto
```



char *argv[] : tableau de pointeurs sur char. argv[argc] vaut toujours NULL.

SYNTHÈSE

À retenir

01

Le système

L'OS sert d'intermédiaire entre les applications et le matériel : processus, mémoire, fichiers, E/S, réseau.

02

Le langage C

C'est le langage des systèmes (UNIX, Linux). La compilation enchaîne cpp → cc1 → as → ld.

03

Compiler proprement

gcc -std=c17 -Wall -Wextra -g. Les avertissements sont des bugs en devenir.

04

Types et tableaux

Les tailles dépendent de la plateforme : utilisez sizeof. Les tableaux n'ont aucun contrôle de bornes.

05

Pointeurs

& donne l'adresse, * l'indirection. Un pointeur est typé, et toujours initialisé (au pire à NULL).

06

Paramètres

Les paramètres sont passés par valeur. Pour modifier une variable, on passe son adresse.

Ressources pédagogiques

Bibliographie

- Systèmes d'exploitation, A. Tanenbaum, Pearson
- La programmation sous UNIX, J.-M. Rifflet, Ediscience
- Unix, Linux et les systèmes d'exploitation, M. Divay, Dunod
- Programmer en langage C, C. Delannoy, Eyrolles

Vidéos

Pile et variables globales / locales en C
Ronan Boulic

Variables et pointeurs en C
Ronan Boulic

Tableau à deux indices et pointeurs
Ronan Boulic

Cours ICC — Wandida EPFL
playlist YouTube

En ligne

Wikilivres : Programmation C

cppreference : la référence du C

Guide ANSSI : règles de programmation sécurisée en C

`Le manuel : man gcc, man 3 printf`

lamae

Questions ?

PROCHAINE SÉANCE

CM 01b — Entrées-sorties et allocation dynamique

stdin / stdout / stderr · printf, scanf, fgets · malloc, calloc, realloc, free

Christophe Cruz · christophe.cruz@ube.fr